



**UNIVERSIDAD
DON BOSCO**

Facultad de Ingeniería
Escuela de Ingeniería en Computación

Asignatura: Programación Orientada a Objetos

Ciclo / Grupo: 01– 2026 / 02L

Desafío 3

Facilitador: Ing. Kevin Jiménez

Elaborado por:

Bryan Ernesto Anaya Brizuela	AB250136
------------------------------	----------

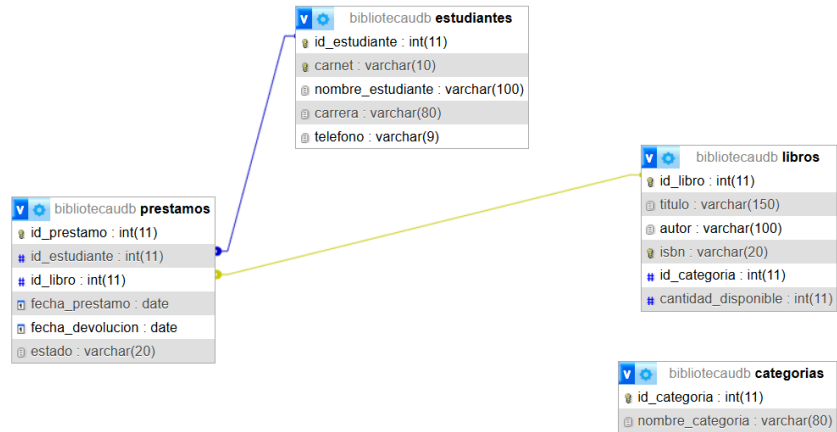
Concepción Getsemaní Miranda Cuéllar	MC250288
--------------------------------------	----------

Fecha de Entrega: Viernes 22 de mayo de 2026

Objetivo del programa

Permite registrar libros, estudiantes y préstamos de una biblioteca utilizando JSP, JavaBeans y MySQL.

1. Creamos la base de datos que vamos a utilizar en el sistema:

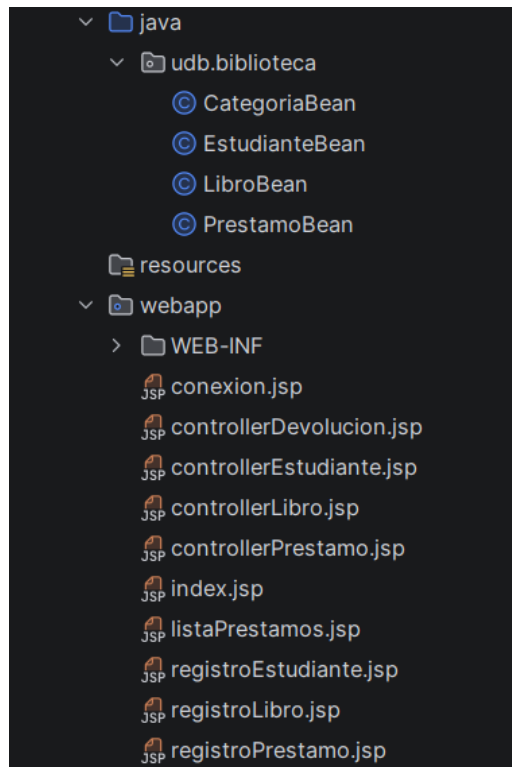


Siendo esta quién posee las 4 tablas a utilizar: categorías, libros, préstamos y estudiantes.

2. Incluimos el driver JDBC de MySQL en el archivo pom.xml como dependencia Maven.

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.0.33</version>
</dependency>
</dependencies>
```

3. Creamos el proyecto en IntelliJ IDEA con sus carpetas beans y JSP correspondientes.



4. Creamos conexión.jsp para poder usar la base de datos (JDBC).

```
1  <!-- Esta página crea la conexión con la base de datos para que los demás JSP la puedan usar -->
2  <%@ page import="java.sql.*" %>
3  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
4  <%
5      Connection conn = null;
6      ResultSet rs = null;
7      PreparedStatement st = null;
8
9      try {
10         Class.forName("com.mysql.cj.jdbc.Driver");
11         conn = DriverManager.getConnection(
12             "jdbc:mysql://localhost/bibliotecaudb" +
13             "?useSSL=false&serverTimezone=America/El_Salvador" +
14             "&allowPublicKeyRetrieval=true",
15             "root", ""
16         );
17     } catch (Exception e) {
18         throw new RuntimeException("Error de conexión: " + e.getMessage());
19     }
20  %>
```

Formularios JSP (Lo que ve el usuario)

registroEstudiante.jsp contiene el formulario utilizado para registrar estudiantes dentro del sistema de biblioteca.

El formulario envía la información ingresada hacia `controllerEstudiante.jsp`, el cual se encarga de procesar los datos y mandarlos al `EstudianteBean.java` para guardarlos en la base de datos.

```
registroEstudiante.jsp x
1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
3  <html lang="es">
10 <body>
24 <div class="container mt-4">
27
28     <!-- Mostrar error si viene redirigido -->
29     <% if (request.getParameter("error") != null) { %>
30     <div class="alert alert-danger"><%= request.getParameter("error") %></div>
31     <% } %>
32
33     <!-- Formulario para registrar estudiantes -->
34     <form action="controllerEstudiante.jsp" method="POST">
35         <div class="mb-3">
36             <label class="form-label">Carnet</label>
37             <input type="text" name="carnet" class="form-control" maxlength="10" required>
38         </div>
39         <div class="mb-3">
40             <label class="form-label">Nombre completo</label>
41             <input type="text" name="nombreEstudiante" class="form-control" required>
42         </div>
43         <div class="mb-3">
44             <label class="form-label">Carrera</label>
45             <input type="text" name="carrera" class="form-control" required>
46         </div>
47         <div class="mb-3">
48             <label class="form-label">Teléfono</label>
49             <input type="text" name="telefono" class="form-control" maxlength="9">
50         </div>
51         <button type="submit" class="btn btn-dark">Registrar</button>
52         <a href="index.jsp" class="btn btn-secondary ms-2">Cancelar</a>
53     </form>
54 </div>
```

registroLibro.jsp contiene el formulario utilizado para registrar libros dentro del sistema de biblioteca.

El formulario envía la información ingresada hacia `controllerLibro.jsp`, el cual se encarga de recibir y procesar los datos para posteriormente enviarlos a `LibroBean.java`, donde son almacenados en la base de datos.

```
registroLibro.jsp x
1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
6  <html lang="es">
13 <body>
27 <!-- Formulario que envía los datos del libro hacia controllerLibro.jsp -->
28 <div class="container mt-4">
29     <h3>Registrar Libro</h3>
30     <hr>
31     <form action="controllerLibro.jsp" method="POST">
32         <div class="mb-3">
33             <label class="form-label">Titulo</label>
34             <input type="text" name="titulo" class="form-control" required>
35         </div>
36         <div class="mb-3">
37             <label class="form-label">Autor</label>
38             <input type="text" name="autor" class="form-control" required>
39         </div>
40         <div class="mb-3">
41             <label class="form-label">ISBN</label>
42             <input type="text" name="isbn" class="form-control">
43         </div>
44         <div class="mb-3">
45             <label class="form-label">Categoria</label>
46             <select name="idCategoria" class="form-select" required>
47                 <option value="">-- Seleccione --</option>
48                 <%
49                     // Cargar categorías dinámicamente desde la BD
50                     CategoriaBean catBean = new CategoriaBean();
51                     List<CategoriaBean> categorias = catBean.getListasCategorias(conn);
52                     for (CategoriaBean c : categorias) {
53
54                 <option value="<%= c.getIdCategoria() %"><%= c.getNombreCategoria() %"></option>
55             </select>
        </div>
    </form>
</div>
</body>
</html>
```

registroPrestamo.jsp permite registrar préstamos de libros dentro del sistema. La información ingresada es enviada a `controllerPrestamo.jsp`, donde posteriormente se procesa y se guarda mediante `PrestamoBean.java`.

```
registroPrestamo.jsp x
1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
7  <html lang="es">
14 <body>
28 <div class="container mt-4">
32   <!-- Mostrar error si el libro no tiene disponibilidad -->
33   <% if (request.getParameter("error") != null) { %>
34     <div class="alert alert-danger"><%= request.getParameter("error") %></div>
35     <% } %>
36
37   <!-- Formulario que manda la información del préstamo al controlador -->
38   <form action="controllerPrestamo.jsp" method="POST">
39
40     <!-- Select dinámico de estudiantes -->
41     <div class="mb-3">
42       <label class="form-label">Estudiante</label>
43       <select name="idEstudiante" class="form-select" required>
44         <option value="">-- Seleccione un estudiante --</option>
45         <%
46           EstudianteBean estBean = new EstudianteBean();
47           List<EstudianteBean> estudiantes = estBean.getListaEstudiantes(conn);
48           for (EstudianteBean e : estudiantes) {
49             <%
50               <option value="<%= e.getIdEstudiante() %>">
51                 <%= e.getCarnet() %> - <%= e.getNombreEstudiante() %>
52               </option>
53             <%
54               }
55             <%
56           </select>
57         </div>
58
59     <!-- Select dinámico de libros (solo los que tienen disponibilidad) -->
```

controllerEstudiante.jsp se encarga de recibir la información enviada desde **registroEstudiante.jsp**.

En este controlador se utilizan instrucciones como `request.getParameter()` para obtener los datos ingresados por el usuario en el formulario.

Posteriormente, los datos son enviados al objeto **EstudianteBean.java** mediante métodos setter, con el objetivo de procesar y almacenar la información en la base de datos.

```

1  <!-- Controlador que procesa el registro de estudiantes -->
2  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
3  <%@ include file="conexion.jsp" %>
4
5  <!-- Crear el bean y asignar automáticamente todos los parámetros del formulario -->
6  <jsp:useBean id="estudiante" class="udb.biblioteca.EstudianteBean" scope="request"/>
7  <jsp:setProperty name="estudiante" property="*" />
8
9  <%
10     // Insertar el estudiante en la base de datos usando el bean
11     try {
12         estudiante.insertar(conn);
13     } catch (Exception e) {
14         // Si hay error (ej: carnet duplicado) redirigir con mensaje
15         response.sendRedirect("registroEstudiante.jsp?error=El carnet ya existe o hubo un error.");
16         return;
17     }
18 %>
19
20 <!DOCTYPE html>
21 <html lang="es">
22 <head>
23     <meta charset="UTF-8">
24     <meta name="viewport" content="width=device-width, initial-scale=1">
25     <title>Estudiante Registrado</title>
26     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
27 </head>
28 <body>
29
30 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
31     <div class="container">
32         <a class="navbar-brand" href="index.jsp">Biblioteca UDB</a>

```

controllerLibro.jsp recibe la información enviada desde registroLibro.jsp. Utiliza `request.getParameter()` para obtener los datos ingresados por el usuario. Posteriormente crea un objeto `LibroBean` y envía la información hacia este para que pueda almacenarse en la base de datos.

```

1  <!-- Controlador encargado de recibir datos del formulario de libros y guardarlos -->
2  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
3  <%@ include file="conexion.jsp" %>
4
5  <!-- Crear el bean y asignar automáticamente todos los parámetros del formulario -->
6  <jsp:useBean id="libro" class="udb.biblioteca.LibroBean" scope="request"/>
7  <jsp:setProperty name="libro" property="*/>
8
9  <%
10     // Insertar el libro en la base de datos usando el bean
11     try {
12         libro.insertar(conn);
13     } catch (Exception e) {
14         // Si hay error (ej: ISBN duplicado) redirigir con mensaje
15         response.sendRedirect("registroLibro.jsp?error=" + e.getMessage());
16         return;
17     }
18 %>
19
20 <!DOCTYPE html>
21 <html lang="es">
22 <head>
23     <meta charset="UTF-8">
24     <meta name="viewport" content="width=device-width, initial-scale=1">
25     <title>Libro Registrado</title>
26     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
27 </head>
28 <body>
29
30 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
31     <div class="container">
32         <a class="navbar-brand" href="index.jsp">Biblioteca UDB</a>

```

controllerPrestamo.jsp se encarga de recibir la información enviada desde registroPrestamo.jsp.

En este controlador se obtienen los datos ingresados por el usuario, como el estudiante, el libro y la fecha del préstamo, utilizando instrucciones como `request.getParameter()`.

Posteriormente, la información es enviada al objeto `PrestamoBean.java` mediante métodos setter, para que pueda procesarse y almacenarse correctamente en la base de datos.

```

1  <!-- Este controlador recibe los datos del préstamo y los manda al Bean para guardarlos -->
2  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
3  <%@ include file="conexion.jsp" %>
4
5  <!-- Crear el bean y asignar automáticamente todos los parámetros del formulario -->
6  <jsp:useBean id="prestamo" class="udb.biblioteca.PrestamoBean" scope="request"/>
7  <jsp:setProperty name="prestamo" property="*/>
8
9  <%
10     // Insertar el préstamo - el bean valida disponibilidad internamente
11     try {
12         prestamo.insertar(conn);
13     } catch (Exception e) {
14         // Si no hay disponibilidad u otro error, redirigir con mensaje
15         response.sendRedirect("registroPrestamo.jsp?error=" + e.getMessage());
16         return;
17     }
18 %>
19
20 <!DOCTYPE html>
21 <html lang="es">
22 <head>
23     <meta charset="UTF-8">
24     <meta name="viewport" content="width=device-width, initial-scale=1">
25     <title>Préstamo Registrado</title>
26     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
27 </head>
28 <body>
29
30 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
31     <div class="container">
32         <a class="navbar-brand" href="index.jsp">Biblioteca UDB</a>

```

listaPrestamos.jsp muestra en una tabla todos los préstamos registrados en el sistema.

La información es obtenida desde **PrestamoBean.java**, el cual consulta los datos almacenados en la base de datos.

```
JSP listaPrestamos.jsp x
1  <%@ page contentType="text/html; charset=UTF-8" language="java" %>
6  <html lang="es">
13 <body>
27 <div class="container mt-4">
35
36     <%
37         // Obtener todos los préstamos desde el bean
38         PrestamoBean prestamoBean = new PrestamoBean();
39         List<PrestamoBean> lista = prestamoBean.getListaPrestamos(conn);
40     %>
41
42     <table class="table table-bordered table-striped">
43         <thead class="table-dark">
44             <tr>
45                 <th>#</th>
46                 <th>Estudiante</th>
47                 <th>Carnet</th>
48                 <th>Libro</th>
49                 <th>Fecha Préstamo</th>
50                 <th>Fecha Devolución</th>
51                 <th>Estado</th>
52                 <th>Acción</th>
53             </tr>
54         </thead>
55         <tbody>
56             <%
57                 for (PrestamoBean p : lista) {
58                     // Determinar el estado visual del préstamo
59                     String estadoVisual = p.getEstadoPrestamo();
60                     String badgeColor = "bg-success";
61                     if (estadoVisual.equals("Vencido")) badgeColor = "bg-danger";
62                     if (estadoVisual.equals("Devuelto")) badgeColor = "bg-secondary";
```

controllerDevolucion.jsp se encarga de procesar la devolución de libros dentro del sistema de biblioteca. Este controlador recibe la información relacionada con el préstamo que será devuelto, utilizando instrucciones como `request.getParameter()` para obtener los datos necesarios.

```
controllerDevolucion.jsp x
1 <!-- Aquí se procesa la devolución de libros prestados -->
2 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
3 <%@ include file="conexion.jsp" %>
4 <%@ page import="udb.biblioteca.PrestamoBean" %>
5 <%
6     // Obtener los parámetros enviados por el link de devolución
7     int idPrestamo = Integer.parseInt(request.getParameter("idPrestamo"));
8     int idLibro    = Integer.parseInt(request.getParameter("idLibro"));
9
10    // Ejecutar la devolución usando el bean
11    try {
12        PrestamoBean prestamoBean = new PrestamoBean();
13        prestamoBean.devolver(conn, idPrestamo, idLibro);
14        // Redirigir a la lista con mensaje de éxito
15        response.sendRedirect("listaPrestamos.jsp?devuelto=true");
16    } catch (Exception e) {
17        response.sendRedirect("listaPrestamos.jsp?error=" + e.getMessage());
18    }
19 %>
20
```

Index.jsp donde el usuario puede acceder a todas las opciones.

```
Index.jsp x
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <!DOCTYPE html>
3 <html lang="es">
4 <head>
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1">
7     <title>Biblioteca UDB</title>
8     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
9 </head>
10 <body>
11 <!-- Página principal desde donde el usuario entra a las demás opciones -->
12
13 <!-- Barra de navegación -->
14 <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
15     <div class="container">
16         <a class="navbar-brand" href="index.jsp">Biblioteca UDB</a>
17         <div class="navbar-nav ms-auto">
18             <a class="nav-link" href="registroLibro.jsp">Registrar Libro</a>
19             <a class="nav-link" href="registroEstudiante.jsp">Registrar Estudiante</a>
20             <a class="nav-link" href="registroPrestamo.jsp">Registrar Préstamo</a>
21             <a class="nav-link" href="listaPrestamos.jsp">Lista de Préstamos</a>
22         </div>
23     </div>
24 </nav>
25
26 <!-- Contenido principal -->
27 <div class="container mt-5">
28     <h2 class="mb-4">Panel Principal</h2>
29     <div class="row">
30         <div class="col-md-3">
31             <div class="card text-center p-3">
32                 <h5>Libros</h5>

```

Beans (almacenar y administrar la información)

EstudianteBean.java contiene la lógica encargada de administrar la información de los estudiantes dentro del sistema de biblioteca.

En este Bean se encuentran los atributos relacionados con los estudiantes, como nombre, apellido, carnet y demás datos necesarios para su registro.

También incluye métodos getter y setter, los cuales permiten recibir y enviar información entre los formularios, controladores y la base de datos.

Además, contiene métodos encargados de realizar operaciones como insertar, consultar, actualizar o eliminar estudiantes mediante instrucciones SQL.

```
EstudianteBean.java x
1 package udb.biblioteca;
2
3 import java.sql.*;
4 import java.util.ArrayList;
5 import java.util.List;
6
7
8 public class EstudianteBean { 16 usages
9
10     private int idEstudiante; 2 usages
11     private String carnet; 3 usages
12     private String nombreEstudiante; 3 usages
13     private String carrera; 3 usages
14     private String telefono; 3 usages
15
16     public EstudianteBean() {} 4 usages
17
18
19 > public int getIdEstudiante() { return idEstudiante; }
22 > public void setIdEstudiante(int idEstudiante) { this.idEstudiante = idEstudiante; }
25
26 > public String getCarnet() { return carnet; }
29 > public void setCarnet(String carnet) { this.carnet = carnet; }
32
33 > public String getNombreEstudiante() { return nombreEstudiante; }
36 > public void setNombreEstudiante(String nombreEstudiante) { this.nombreEstudiante = nombreEstudiante; }
39
40 > public String getCarrera() { return carrera; }
43 > public void setCarrera(String carrera) { this.carrera = carrera; }
46
47 > public String getTelefono() { return telefono; }
50 > public void setTelefono(String telefono) { this.telefono = telefono; }
```

LibroBean.java contiene la lógica encargada de administrar la información de los libros.

En este Bean se encuentran atributos, métodos getter y setter, así como métodos para insertar, listar, actualizar o eliminar registros en la base de datos.

```

7      public class LibroBean { 16 usages
64
65      >      public CategoriaBean getCategoria() { return categoria; }
68      >      public void setCategoria(CategoriaBean categoria) { this.categoria = categoria; }
71
72      // Métodos de lógica de negocio
73
74      /*
75      Retorna el nombre de la categoría asociada al libro.
76      Útil para mostrarlo directamente en las vistas JSP.
77      @return nombre de la categoría o cadena vacía si no hay categoría asignada
78      */
79      public String getNombreCategoria() { no usages
80          if (categoria != null) {
81              return categoria.getNombreCategoria();
82          }
83          return "";
84      }
85
86      /*
87      Retorna todos los libros almacenados en la base de datos,
88      incluyendo el nombre de su categoría mediante un JOIN.
89      @param conn Conexión JDBC activa
90      @return Lista de objetos LibroBean
91      @throws SQLException si ocurre un error al consultar la base de datos
92      */
93
94      // Obtiene todos los libros guardados para mostrarlos en la página
95      public List<LibroBean> getListLibros(Connection conn) throws SQLException { 1 usage
96          List<LibroBean> lista = new ArrayList<>();
97
98          String sql = "SELECT l.id libro, l.titulo, l.autor, l.isbn, " +

```

CategoriaBean.java contiene la lógica encargada de administrar las categorías de los libros dentro del sistema de biblioteca.

En este Bean se encuentran los atributos relacionados con las categorías, como el identificador y el nombre de la categoría.

También incluye métodos getter y setter, los cuales permiten recibir y enviar información entre los formularios, controladores y la base de datos.

Además, contiene métodos encargados de realizar operaciones como insertar, consultar o listar categorías mediante instrucciones SQL.

```

9      public class CategoriaBean { 15 usages
34          // Métodos de lógica de negocio
35
36          /*
37              Retorna todas las categorías almacenadas en la base de datos.
38              @param conn Conexión JDBC activa (obtenida desde conexion.jsp)
39              @return Lista de objetos CategoriaBean
40              @throws SQLException si ocurre un error al consultar la base de datos
41          */
42
43          // Este metodoo obtiene todas las categorías de la BD y las manda a donde se necesiten mostrar
44          public List<CategoriaBean> getListasCategorias(Connection conn) throws SQLException { 1 usage
45              List<CategoriaBean> lista = new ArrayList<>();
46
47              String sql = "SELECT id_categoria, nombre_categoria FROM categorias ORDER BY nombre_categoria";
48
49              try (PreparedStatement ps = conn.prepareStatement(sql);
50                  ResultSet rs = ps.executeQuery()) {
51
52                  while (rs.next()) {
53                      CategoriaBean cat = new CategoriaBean();
54                      cat.setIdCategoria(rs.getInt( columnLabel: "id_categoria"));
55                      cat.setNombreCategoria(rs.getString( columnLabel: "nombre_categoria"));
56                      lista.add(cat);
57                  }
58              }
59              return lista;
60          }
61
62          /*
63              Inserta una nueva categoría en la base de datos.
64              @param conn Conexión JDBC activa

```

PrestamoBean.java contiene la lógica encargada de administrar los préstamos de libros dentro del sistema de biblioteca.

En este Bean se encuentran los atributos relacionados con los préstamos, como el estudiante, el libro, la fecha de préstamo y otros datos necesarios para registrar la operación.

También incluye métodos getter y setter, los cuales permiten recibir y enviar información entre los formularios, controladores y la base de datos.

Además, contiene métodos encargados de realizar operaciones como registrar préstamos, consultar información, listar préstamos y actualizar estados mediante instrucciones SQL.

```
9      public class PrestamoBean { 13 usages
88          Determina si el préstamo está vigente o vencido comparando la fecha
89          de devolución con la fecha actual del sistema.
90          Solo aplica a préstamos con estado 'Activo'.
91          @return "Devuelto" si el préstamo fue devuelto,
92                  "Vigente".....si la fecha de devolución es hoy o futura,
93                  "Vencido".....si la fecha de devolución ya pasó y aún no fue devuelto.
94      */
95      public String getEstadoPrestamo() { 1 usage
96          if ("Devuelto".equalsIgnoreCase(this.estado)) {
97              return "Devuelto";
98          }
99          if (this.fechaDevolucion == null || this.fechaDevolucion.isEmpty()) {
100              return "Sin fecha";
101          }
102
103          LocalDate hoy = LocalDate.now();
104          LocalDate devolucion = LocalDate.parse(this.fechaDevolucion);
105
106          return devolucion.isBefore(hoy) ? "Vencido" : "Vigente";
107      }
108
109      /*
110      Registra un nuevo préstamo en la base de datos.
111      Antes de insertar verifica que el libro tenga unidades disponibles;
112      si no las tiene lanza una excepción para evitar el préstamo.
113      Al insertar, descuenta una unidad del stock del libro.
114      @param conn Conexión JDBC activa
115      @throws SQLException.....si ocurre un error de base de datos
116      @throws Exception.....si el libro no tiene unidades disponibles
117      */
118  }
```